

SCS 1309

Database Management Systems

Lecture 1 – Structured Query Language (SQL) –Part 1

MVPT Lakshika,
Lecturer (Probationary),
Department of Information Systems Engineering, UCSC.
Contact: tlv@ucsc.cmb.ac.lk

Course Description

Number of Credits: 4 Credits (3L,1P)

Evaluation Criteria: Continuous Assessments: 40%, Final Examination: 60%

This lecture series will cover,

- **ILO4:** Apply normalization techniques up to the 4th normal form to ensure data integrity and minimize redundancy in database design.
- **ILO5:** Utilize relational algebra and calculus to perform operations such as selection, projection, joins, division, set operations, and renaming on relational data.
- **ILO6:** Demonstrate proficiency in SQL, including writing and executing basic to complex SQL statements, understanding aggregation operators, nested queries, handling null values, and implementing views, relations, stored procedures, and triggers.
- **ILO7:** Apply critical thinking and problem-solving skills to real-world scenarios involving database management, including identifying and resolving issues related to database design and querying.

Recommended Textbooks

- Elmasri, R. and Navathe, S.B., 2015. *Fundamentals of database systems*. 7th Edition, Addison-Wesley.
- Ramakrishnan, R. and Gehrke, J., 2003. *Database management systems* (Vol. 3). New York: McGraw-Hill.
- DeBarros, A., 2022. *Practical SQL: A Beginner's Guide to Storytelling with Data*. No Starch Press.

Practical Sessions

Software Tools:

- Windows Command-Line Interface (CLI)
- Microsoft Access
- MS SQL Server

Assessment Criteria

- One (1) In-class Assessment
- One (1) Practical Assessment

(Dates will be announced)

- Several Unannounced In-class quizzes
- Several Unannounced practical quizzes

The selected list of unannounced quizzes will be considered as the make-up assessment.

✗ No make-up assessments at the semester end!

Course Content

1. Structured Query Language

- Basic syntax and execution of SQL statements
- Union, intersection, and except
- Aggregation operators
- Nested queries
- Null values
- Views
- Deleting and Altering relations and views
- Complex integrity constraints
- Stored procedures
- Triggers

Course Content

2. Relational Algebra and Calculus

- Selection and Projection
- Joins and Division
- Set Operations and Renaming
- Relational Calculus

3. Database Normalization

- Introduction to data normalization and normal forms
- UNF, 1NF, 2NF, 3NF, BCNF
- Normalization considerations

Outline of this session...

1. Introduction to SQL
2. SQL Basics
3. ISO SQL Data Types
4. ISO SQL Scalar Operators

1. Introduction to Structured Query Language (SQL)

What is SQL?

- The Structured Query Language (SQL), widely recognized as SQL, originated from the evolution of the relational model.
 - Designed to meet the fundamental requirements of a database language.
- SQL **isn't a procedural language**, as are FORTRAN, BASIC, C, COBOL, Pascal, and Java.
 - In procedural languages, one requires writing a procedure (a linear sequence or loop) that performs one specific operation after another until the task is complete. The programmer specifies the execution order.
- SQL is **nonprocedural**.
 - In a nonprocedural language, simply tell *what you want* instead of telling the system how to get you what you want.
 - The DBMS decides the best way to get you what you request.

What is SQL?

- What does it mean by ‘tell the system what you want’?

e.g., Consider the **EMPLOYEE** table. You want to retrieve all the rows from that table that correspond to senior people, defined as *anyone older than 40 or anyone earning more than 150,000 LKR per month*.

- You can make the desired retrieval by using the following **query**:

```
SELECT * FROM EMPLOYEE WHERE Age>40 OR Salary>150000 ;
```

- This statement retrieves all rows from the **EMPLOYEE** table where either the value in the **Age** column is greater than 40 or the value in the **Salary** column is greater than 150,000.

What is SQL?

- SQL commands use intuitive **English-like** syntax.

e.g., **CREATE TABLE** Staff (staffNo **VARCHAR**(5), IName **VARCHAR**(15), salary **DECIMAL**(7,2));

INSERT INTO Staff **VALUES** ('SG16', 'Brown', 8300);

SELECT staffNo, IName, salary **FROM** Staff **WHERE** salary > 10,000;

- SQL is versatile and can be used by **various professionals**, including:
 - Database Administrators (DBA): For managing and maintaining databases.
 - Application Developers: For building data-driven applications.
 - Management Personnel: For retrieving critical business insights.
 - End Users: For ad-hoc data queries.

e.g.; Business analysts, Account officers, HR officers, Research assistants, Data entry supervisors

Objectives of SQL

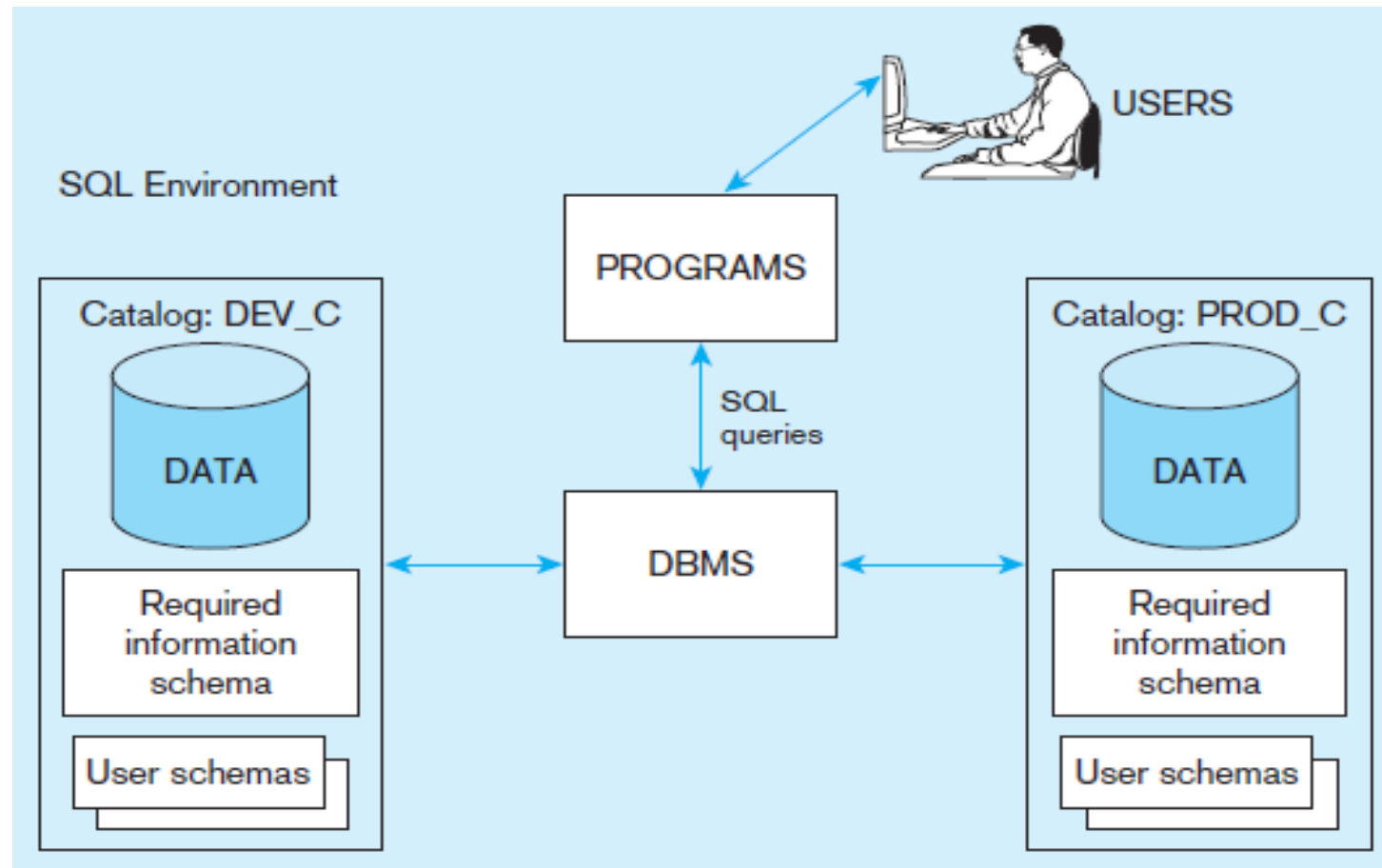
- Ideally, a database language **should allow** a user to:
 - Create the database and relation structures.
 - Perform basic data management tasks(e.g., insertion, modification, and deletion of data from the relations).
 - Perform both simple and complex queries.
- A database language must perform these tasks with **minimal user effort**.
 - The command structure and syntax must be relatively easy to learn.
- A database language **must be portable**.
 - It must conform to some recognized standard so that we can use the same command structure and syntax when we move from one DBMS to another.

A (Very) Little History

- Originally, SQL was called SEQUEL (**S**tructured **E**nglish **Q**Uery **L**anguage).
 - Designed & implemented at IBM Research labs as the interface for an experimental relational DBMS (early 1970s).
 - In 1986, the American National Standards Institute (ANSI) established a standard for SQL, which was later internationally recognized by the International Organization for Standardization (ISO) in 1987.
 - SQL-86(SQL1) → SQL-92 (SQL2) → SQL:1999(SQL3) → [SQL:2003](#) → SQL:2006 → SQL: 2008 → SQL:2011
- Over time, SQL has become the standard and dominant language for commercial relational databases.
 - Supported by DBMSs across a variety of hardware platforms, from personal computers to mainframes.

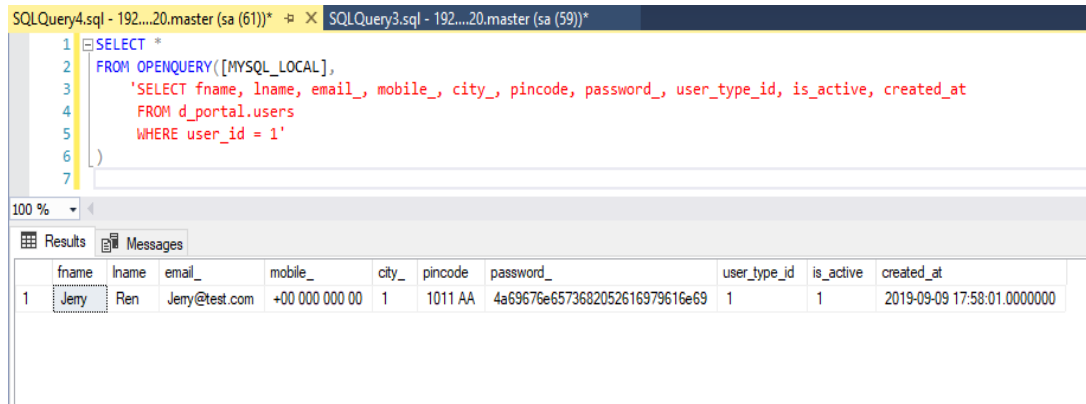
Typical SQL environment

- A simplified schematic of a typical SQL environment.



How to extract information using SQL?

- SQL can be utilized in **two distinct** ways:
 - **Interactive Mode:** Users directly enter SQL statements through a terminal interface.



The screenshot shows a SQL query editor with two tabs: 'SQLQuery4.sql - 192....20.master (sa (61))' and 'SQLQuery3.sql - 192....20.master (sa (59))'. The active tab contains the following SQL query:

```
1 SELECT *
2 FROM OPENQUERY([MYSQL_LOCAL],
3 'SELECT fname, lname, email_, mobile_, city_, pincode, password_, user_type_id, is_active, created_at
4 FROM d_portal.users
5 WHERE user_id = 1'
6 )
7
```

Below the query editor, there is a 'Results' tab showing a table with 10 columns: fname, lname, email_, mobile_, city_, pincode, password_, user_type_id, is_active, and created_at. The table contains one row of data for user_id = 1.

	fname	lname	email_	mobile_	city_	pincode	password_	user_type_id	is_active	created_at
1	Jerry	Ren	Jerry@test.com	+00 000 000 00	1	1011 AA	4a69676e6573682052616979616e69	1	1	2019-09-09 17:58:01.0000000

- **Embedded Mode:** SQL commands are integrated into general-purpose programming languages such as Java or C/C++.

```
/**
 * insert multiple actors
 */
public void insertActors(List<Actor> list) {
    String SQL = "INSERT INTO actor(first_name,last_name) "
        + "VALUES(?,?)";
    try (
        Connection conn = connect();
        PreparedStatement statement = conn.prepareStatement(SQL);) {
        int count = 0;

        for (Actor actor : list) {
            statement.setString(1, actor.getFirstName());
            statement.setString(2, actor.getLastName());
```

Classification of SQL commands

- SQL is an example of a **transform-oriented** language – A language designed to use relations to transform inputs into required outputs.
- The ISO SQL standard defines two major components:
 - **Data Definition Language (DDL)**: for defining and modifying the **database structure or schema**.
 - **Data Manipulation Language (DML)**: for retrieving and modifying **data**.
- Additionally, SQL includes:
 - **Data Control Language (DCL)**: for managing **user permissions and access**.
 - **Transaction Control Language (TCL)**: Ensures the consistency and reliability of **database transactions** (a series of operations performed within a DBMS).

What Is RDBMS?

- A **relational database management system (RDBMS)** is a collection of programs and capabilities that enable IT teams and others to create, update, administer and otherwise interact with a relational database.
- In other words, **RDBMS is a program used to create, update, and manage data in tables (relations).**
- RDBMSes store data in the form of **tables**, with most commercial relational database management systems using SQL to access the database.
- However, since SQL was invented after the initial development of the relational model, it is not necessary for RDBMS use.

Examples of RDBMS

- Some of the most well-known RDBMSs include
 - MySQL, PostgreSQL, MariaDB, Microsoft SQL Server, and Oracle Database



- Cloud-based relational databases include

-Cloud SQL, Cloud Spanner and AlloyDB, Amazon Aurora



Cloud SQL

Google
Cloud
Spanner



AlloyDB



2. SQL BASICS

SQL commands

- SQL commands perform **data definition** functions, **data-manipulation** functions and **data-control** functions.
- An SQL statement is composed of the following elements:
 - **Reserved Words:** Fixed parts of the SQL language with fixed meanings. They must be spelt **exactly** as defined and cannot be split across multiple lines.
 - **User-Defined Words:** Made up by the user, adhering to specific **syntax rules**. They represent the names of various database objects, such as tables, columns, views, and indexes.
 - **Statement Terminator:** SQL statements end with a statement terminator, often a semicolon (;)

*e.g., SELECT * FROM EMPLOYEE WHERE Age>40 OR Salary>150000 ;*

*e.g., **SELECT** * **FROM** **EMPLOYEE** **WHERE** **Age**>40 **OR** **Salary**>150000 ;*

Writing SQL commands

- Most SQL components are case-insensitive.
 - Can be typed either upper or lower case (e.g., `SELECT` and `select` are treated the same).
- Literal character data must match the database exactly.
 - e.g., Store a person's surname as `'SMITH'` and then search for it using the string `'Smith'`, the row will not be found.
- Readability: While SQL is free-format, thus doesn't need specific alignment.
 - But, proper indentation and formatting improve the readability and maintainability of SQL statements.

SQL:2003 Commands

- **DDL related:** CREATE, ALTER, DROP TABLE, COLUMN, INDEX, CONSTRAINT, VIEW
- **DML related:** SELECT, INSERT, UPDATE, DELETE, FROM, WHERE, GROUP BY, ORDER BY, HAVING, JOIN
- **Data Types:** INT, INTEGER, SMALLINT, BIGINT, CHAR, VARCHAR, NCHAR, NCLOB, BLOB, BOOLEAN, DATE, TIME, TIMESTAMP, DECIMAL, NUMERIC, FLOAT, REAL, DOUBLE
- **Control Flow and Transaction Management:** BEGIN, COMMIT, ROLLBACK, CASE, WHEN, THEN, ELSE, END
- **Functions and Operators:** AVG, COUNT, MAX, MIN, SUM, COALESCE, NULLIF, CAST, CONVERT, AND, OR, NOT, LIKE, IN, BETWEEN
- **Newer Features introduced or enhanced in SQL:2003:** WINDOW, CUBE, ROLLUP, MERGE, XML, CALLED, RECURSIVE

3. ISO SQL DATA TYPES

SQL Data Definition and Data Types

- SQL uses the terms **table**, **row**, and **column** for the formal relational model terms *relation*, *tuple*, and *attribute*, respectively.

Diagram illustrating the mapping of SQL terms to formal relational model terms:

- relation name** points to the table name **Student**.
- attribute name** points to the column names: **ID**, **Name**, **Address**, **Phone**, and **GPA**.
- tuple** points to a specific row in the table.

ID	Name	Address	Phone	GPA
12345	John D	123 My Ln	123-4567	3.5
56789	Jane D	321 YourWay	876-5678	3.8

- The main SQL command for data definition is the **CREATE** statement.
 - Used to create schemas, tables (relations), types, domains and other constructs such as views, assertions, and triggers.
- SQL specification is very large!

SQL Identifiers

- SQL identifiers are used to **identify objects in the database, such as table names, view names, and columns.**
- The characters that can be used in a user-defined SQL identifier must appear in a character set.
- The ISO standard provides a default character set, which consists of the uppercase letters A . . . Z, the lowercase letters a . . . z, the digits 0 . . . 9, and the underscore (_) character.
- It is also possible to specify an alternative character set.
- The following restrictions are imposed on identifiers.
 - An identifier can be **no longer than 128 characters** (most dialects have a much lower limit than this).
 - An identifier **must start with a letter** (not a number)
 - An identifier **cannot contain spaces.**

ISO SQL Data types

- The basic **data types** available for attributes include:
 - Numeric
 - String (Character string & Bit string)
 - Boolean
 - Temporal
- Within each of these general types may be several subtypes.
e.g., numeric data types: Exact Numeric and Approximate Numeric

SCS 1309

Database Management Systems

Lecturer 2 – Structured Query Language (SQL) –Part 2

MVPT Lakshika,
Lecturer (Probationary),
Department of Information Systems Engineering, UCSC.
Contact: tlv@ucsc.cmb.ac.lk

Outline of this session...

1. ISO SQL Data Types
2. ISO SQL Scalar Operators

ISO SQL Data types

- The basic **data types** available for attributes include:
 - Numeric
 - String (Character string & Bit string)
 - Boolean
 - Temporal
- Within each of these general types may be several subtypes.
e.g., numeric data types: Exact Numeric and Approximate Numeric

ISO SQL Data types - Numeric

- Two categories of numeric data types: **Exact Numeric** and **Approximate Numeric**.

1. Exact Numeric data types (5) include:

- Integer numbers of various sizes (**INTEGER** or **INT**, **SMALLINT**, **BIGINT**)
- Formatted numbers declared using **DECIMAL(*p*, *s*)** or **DEC(*p*, *s*)**, **NUMERIC(*p*, *s*)**
 - ***p*** = *precision* (maximum number of digits the number can have) and ***s*** = *scale* (number of digits after the decimal point)
 - The default for scale is zero, and the default for precision is implementation-defined.

2. Approximate Numeric data types (3) include:

- Floating-point (real) numbers of various precision (**FLOAT**, **REAL**, and **DOUBLE PRECISION**)

ISO SQL Data types - Exact Numeric data

1. Exact Numeric Data Types:

1.1. SMALLINT

- Used for small positive or negative whole numbers without a fractional part.
- A smaller version of INTEGER.
- Requires less storage and supports a smaller range of values (usually 16-bit integers)

e.g., `CREATE TABLE Employee (Age SMALLINT);`

`INSERT INTO Employee (Age) VALUES (25);`

1.2. INTEGER (or INT)

- Used for large positive or negative whole numbers without a fractional part.
- Supports a range of values (typically 32-bit integers)

e.g., `CREATE TABLE Employee (EmployeeID INTEGER);`

`INSERT INTO Employee (EmployeeID) VALUES (123);`

ISO SQL Data types - Exact Numeric data

1. Exact Numeric Data Types:

1.3. BIGINT

- Used for very large whole numbers.
- Supports a wider range of values (usually 64-bit integers).

e.g., `CREATE TABLE Employee (Population BIGINT);`

`INSERT INTO Employee (Population) VALUES (9876543210);`

ISO SQL Data types - Exact Numeric data

Summary of Integer Types:

Type	Storage (Bytes)	Signed Range	Unsigned Range (MySQL/MariaDB extension)	Standard SQL?
TINYINT	1	-128 to 127	0 to 255	No
SMALLINT	2	-32,768 to 32,767	0 to 65,535	Yes
INTEGER/INT	4	-2,147,483,648 to 2,147,483,647	0 to 4,294,967,295	Yes
BIGINT	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	0 to 18,446,744,073,709,551,615	Yes

ISO SQL Data types - Exact Numeric data

1. Exact Numeric Data Types:

1.4. NUMERIC [precision [, scale]]

- Have a fractional component in addition to its integer component.

e.g., `CREATE TABLE Employee ( Salary NUMERIC(5, 2));`

✓ 123.45 ✗ 12345.67

1.5. DECIMAL [precision [, scale]] (or DEC [precision [, scale]])

- Similar to NUMERIC, but implementations may allow slightly more flexibility in storage.

e.g., `CREATE TABLE Employee ( Salary DECIMAL(7, 2));`

✓ 12345.67 ✗ 1234567.89

NUMERIC [precision [, scale]]

*total number of digits
the number can have*

*total number of
decimal places*

ISO SQL Data types - Approximate Numeric Data

- Used for defining numbers that do not have an exact representation, such as floating-point numbers that **represent values with a close approximation rather than an exact value.**
- Quantities have such a large range of possible values (many orders of magnitude).
- Data types: **FLOAT [precision], REAL, DOUBLE PRECISION**
- Notation is similar to scientific notation, in which a number is written as a mantissa times some power of ten (the exponent).

e.g., 10E3 (10×10^3), +5.2E6 ($+5.2 \times 10^6$), -0.2E-4 (-0.2×10^{-4})

ISO SQL Data types - Approximate Numeric Data

2. Approximate Numeric Data Types:

2.1. FLOAT

- A general floating-point data type.
- Can store a wide range of values (**extremely large** or **extremely small** numbers), with precision dependent on the implementation or specified precision.

e.g., `CREATE TABLE Employee (Measurement FLOAT(10));`

`INSERT INTO Employee (Measurement) VALUES (12345.678);`

Here, `FLOAT(10)`: **(10)** refers to the **p**: how many bits to use for the "mantissa" (significant part of the number).

ISO SQL Data types - Approximate Numeric Data

2. Approximate Numeric Data Types:

2.2. REAL (*equivalent to* FLOAT(24))

- A **single-precision** floating-point type (**4 bytes** of storage)
- Typically, equivalent to FLOAT with a predefined precision (implementation-dependent, often 24 bits: **REAL**).
- Range approximately between -3.4×10^{38} to $+3.4 \times 10^{38}$ (-3.4E+38 to 3.4E+38)

e.g., CREATE TABLE *Employee* (*Temperature* **REAL**);

INSERT INTO *Employee* (*Temperature*) VALUES (36.6);

2.3. DOUBLE PRECISION (*equivalent to* FLOAT(53))

- A **double-precision** floating-point type (**8 bytes** of storage)
- Provides more precision compared to REAL (implementation-dependent, often 53 bits).
- Range approximately between -1.79×10^{308} to $+1.79 \times 10^{308}$ (-1.8E+308 to 1.8E+308)

ISO SQL Data types

Activity 1

Looking at the **DreamHome** relational schema provided, identify which data type would be best suited for the following columns.

Salary, rent, rooms, maxRent

Column	Recommended Type	Reason
salary	DECIMAL(10, 2)	Stores currency values; requires precision for accounting
rent	DECIMAL(8, 2)	Stores monthly costs; similar to salary, precision is key
rooms	INTEGER	You cannot have a fraction of a room; it is a whole number
maxRent	DECIMAL(8, 2)	Represents a ceiling for financial transactions

ISO SQL Data types - Character data

- **Character** data consists of a sequence of characters from an implementation-defined character set defined by the vendor of the particular SQL dialect.
- Thus, the exact characters that can appear as data values in a character type column will vary.
- **ASCII** and **EBCDIC** are two sets in common use today.
- Three main types of character data:
 - fixed character data (**CHARACTER** or **CHAR**)
 - Varying character data (**CHARACTER VARYING** or **VARCHAR**)
 - Character large object data (**CHARACTER LARGE OBJECT** or **CLOB**)
- Also have three variants of these types of character data: **NATIONAL CHARACTER**, **NATIONAL CHARACTER VARYING**, and **NATIONAL CHARACTER LARGE OBJECT**.
 - Function same as the **CHAR**, **VARCHAR**, and **CLOB** data types, except that the character set you're specifying is different from the default character set. You can specify the character set as you define a table column.

ISO SQL Data types - Character data

Character Data Types:

1. CHARACTER(*x*) or CHAR(*x*)

- *x* is the number of characters.

e.g., **branchNo CHAR(4)**: **branchNo** of the **Branch** table, which has a **fixed length** of four characters (the maximum length of any data is 4 characters)

- If you **don't specify an argument** (no value in place of the *x*), the field length is **one character**.
- If you enter data with **fewer characters** than the specified number, SQL fills the remaining character spaces with blanks on the right to make up the required size.

ISO SQL Data types - Character data

Character Data Types:

2. CHARACTER VARYING (x) or VARCHAR (x)

- **x** is the **maximum number** of characters permitted.

e.g., **address VARCHAR(30): address** of the **PrivateOwner** table, which has a **variable number of characters** up to a maximum of 30.

- Useful if entries in a column can vary in length, but you don't want SQL to pad the field with blanks.
- This data type stores exactly the number of characters that the user enters.
- **No default value** exists for this data type.

ISO SQL Data types - Character data

Character Data Types:

3. CHARACTER LARGE OBJECT (CLOB)

- Introduced with SQL:1999.
- Used with **huge character strings** that are too large for the CHAR type.
- CLOBs behave much like ordinary character strings, but there are **several restrictions** on what you can do with them.
 - A CLOB may not be used in a PRIMARY KEY, FOREIGN KEY, or UNIQUE predicate.
 - It may not be used in a comparison other than one for either equality or inequality (Because of their large size, applications generally do not transfer CLOBs to or from a database).
 - Instead, a special client-side type called a *CLOB locator* is used to manipulate the CLOB data.
 - It is a parameter whose value identifies a character large object.

ISO SQL Data types - BIT DATA

- The **BIT** data type is used to define *bit* strings, that is, a sequence of binary digits (bits), each having either the value 0 or 1.
- The format for specifying the bit data type is similar to that of the character data type.

e.g., To hold the fixed-length binary string "0011", you can declare a column *bitString*, as:

bitString **BIT**(4)

ISO SQL Data types - BIT DATA

Activity 2

Based on the same schema, identify which data type would be best suited for the following columns.

branchNo, sex, fName, address, staffNo

Column	Recommended Type	Reason
branchNo	CHAR(4)	Consistently 4 characters (e.g., B005, B007)
sex	CHAR(1)	Always a single letter ('M' or 'F')
fName	VARCHAR(15)	Names vary in length (e.g., 'Ann' vs. 'Julie')
address	VARCHAR(50)	Street addresses can be very short or very long
staffNo	CHAR(4)	Employee codes such as 'SL21' or 'SG37' follow a strict 4-character format.

ISO SQL Data types – Temporal Data

- The **datetime data** type is used to define points in **time** to a certain degree of accuracy.
- The ISO standard subdivides the datetime data type into YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, **TIMEZONE_HOUR**, and **TIMEZONE_MINUTE**.
 - The latter two fields specify the hour and minute part of the **time zone** offset from Universal Coordinated Time (which used to be called Greenwich Mean Time).
- Three main types:

```
DATE  
TIME [timePrecision] [WITH TIME ZONE]  
TIMESTAMP [timePrecision] [WITH TIME ZONE]
```
- The **timePrecision** is the number of decimal places of accuracy to which the **SECOND** field is kept.
- If not specified,
 - **TIME** defaults to a precision of **0** (that is, whole seconds)
 - **TIMESTAMP** defaults to **6** (that is, microseconds)
- **WITH TIME ZONE** keyword controls the presence of the **TIMEZONE_HOUR** and **TIMEZONE_MINUTE** fields.

ISO SQL Data types – Temporal Data

- **DATE** data type is used to store calendar dates.
e.g., `viewDate DATE`
 - Components are YEAR, MONTH, and DAY in the form **YYYY-MM-DD** (ten positions) (e.g., '2014-09-27')
- **TIME** data type is used to store time.
e.g., `loginTime TIME`
 - Components are HOUR, MINUTE, and SECOND in the form **HH:MM:SS** (eight positions) (e.g., '09:12:47')
- **TIMESTAMP** is used to store both dates and times.
e.g., `logTime TIMESTAMP`
 - Components are **DATE** and **TIME** fields, plus a minimum of six positions for decimal fractions of seconds (e.g., '2014-09-27 09:12:47.648302')

ISO SQL Data types - Temporal Data

- Only valid dates and times should be allowed by the SQL implementation.
e.g., months should be between 1 and 12, days must be between 01 and 31
- The < (less than) comparison can be used with dates or times.
 - An earlier date is considered to be smaller than a later date, and similarly with time.
- Literal values are represented by single-quoted strings preceded by the keyword.

e.g., CREATE TABLE *Employee* (viewDate **DATE**);

INSERT INTO *Employee* (viewDate) VALUES ('2014-09-27');

CREATE TABLE *Employee* (loginTime **TIME**);

INSERT INTO *Employee* (loginTime) VALUES ('09:12:47');

ISO SQL Data types - Temporal Data

- **INTERVAL** data type is used to **represent periods of time**.
 - MySQL does not have a built-in INTERVAL data type.
 - But supports INTERVAL in expressions to manipulate dates and times.
- Every interval data type consists of a **subset of the fields**: YEAR, MONTH, DAY, HOUR, MINUTE, SECOND.
- **Two classes** of interval data type:
 - The **year–month** class may contain only the YEAR and/ or the MONTH fields (YYYY-MM)
 - The **day–time** class may contain only a contiguous selection from DAY, HOUR, MINUTE, SECOND (DD HH:MM:SS).

```
INTERVAL { {startField TO endField} singleDatetimeField}  
startField = YEAR | MONTH | DAY | HOUR | MINUTE  
            [(intervalLeadingFieldPrecision)]  
endField   = YEAR | MONTH | DAY | HOUR | MINUTE | SECOND  
            [(fractionalSecondsPrecision)]  
singleDatetimeField = startField | SECOND  
                    [(intervalLeadingFieldPrecision [, fractionalSecondsPrecision])] 22
```

ISO SQL Data types - Temporal Data

- In all cases, *startField* has a leading field precision that defaults to 2.

e.g., **INTERVAL YEAR(2) TO MONTH**

- represents an interval of time with a value between 0 years 0 months, and 99 years 11 months (e.g., "2 years and 6 months")

e.g., **INTERVAL HOUR TO SECOND(4)**

- represents an interval of time with a value between 99 hours 59 minutes 59.9999 seconds (the fractional precision of second is 4).

- If the precision is **not explicitly specified**, the default behavior is determined by the database implementation, which is commonly **2 digits**.

ISO SQL Data types - Temporal Data

Activity 3

Based on the same schema, identify which data type would be best suited for the following columns.

DOB, viewDate

Column Name	Recommended Type	Reason
DOB	DATE	Stores day, month, and year of birth (e.g., '1-Oct-45')
viewDate	DATE	Records the specific day a client visited a property (e.g., '24-May-04')

ISO SQL Data types - Boolean data

- **Boolean** data consists of the distinct truth values **TRUE** and **FALSE**.
- In SQL, **TRUE** is considered **greater than FALSE** ($\text{TRUE} > \text{FALSE}$).
- Unless prohibited by a NOT NULL **constraint**, Boolean data also supports the **UNKNOWN** truth value as the **NULL** value.

e.g., Analogy: "Is it raining outside?"

True: You see rain, **False:** You see the sun, **Unknown:** You are in a room with no windows.

- When a Boolean column contains NULL, logical operations involving it follow **three-valued logic (3VL)**:
 - TRUE **AND** UNKNOWN → UNKNOWN
 - FALSE **AND** UNKNOWN → FALSE
 - TRUE **OR** UNKNOWN → TRUE
 - FALSE **OR** UNKNOWN → UNKNOWN
 - NOT TRUE → FALSE
 - NOT FALSE → TRUE
 - NOT UNKNOWN → UNKNOWN

ISO SQL Data types - Boolean data

e.g., CREATE TABLE user_accounts (user_id SERIAL PRIMARY KEY,

is_active BOOLEAN,

is_verified BOOLEAN NOT NULL DEFAULT FALSE,

marketing_opt_in BOOLEAN DEFAULT NULL);

- ***is_active***: This is a "Three-Valued Logic" (3VL) column. It can be True (Active), False (Deactivated), or Null (Status Pending).
- ***is_verified***: Force the database to only accept TRUE or FALSE. DEFAULT FALSE: This ensures that if you don't provide a value, SQL automatically picks FALSE instead of leaving it as NULL.

Activity 4

If you have given a list of students and you filter for WHERE grade > 50, will you see the students who missed the exam (NULL grade)?

Answer: **No**, because NULL > 50 is Unknown, and Unknown is not True.

ISO SQL Data types - Summary

- The ISO standard provides eight base data types.

Data type	Declarations				
boolean	BOOLEAN				
character	CHAR	VARCHAR			
bit [†]	BIT	BIT VARYING			
exact numeric	NUMERIC	DECIMAL	INTEGER	SMALLINT	BIGINT
approximate numeric	FLOAT	REAL	DOUBLE PRECISION		
datetime	DATE	TIME	TIMESTAMP		
interval	INTERVAL				
large objects	CHARACTER LARGE OBJECT		BINARY LARGE OBJECT		

[†] BIT and BIT VARYING have been removed from the SQL:2003 standard.

ISO SQL SCALAR Operators

- SQL provides several **built-in scalar operators** and **functions** that can be used to construct a scalar expression (an expression that evaluates to a **scalar or a single value**)
- Scalar operators:
 - Arithmetic operators: +, -, *, /
 - Comparison: =, >, <, !=
 - Concatenation: ||
 - Logical: AND, OR, NOT
 - e.g.,
- Some other operators:
 - CHAR_LENGTH:** Returns the length of a string in characters. For example, **CHAR_LENGTH('Beech')** returns 5.
 - LOWER, UPPER, CURRENT_DATE, CURRENT_TIME, TRIM**

Commenting

- Comments in SQL are similar to comments in other programming languages, such as Java, C++, Python, etc.
- They are primarily used to define a code section for easier understanding.
- Comments can be,

- Single line

```
--Select all:  
SELECT * FROM Customers;
```

- Inline

```
SELECT * FROM Customers -- WHERE City='Berlin';
```

- Multi-line

```
/*Select all the columns  
of all the records  
in the Customers table:*/  
SELECT * FROM Customers;
```

Any questions?